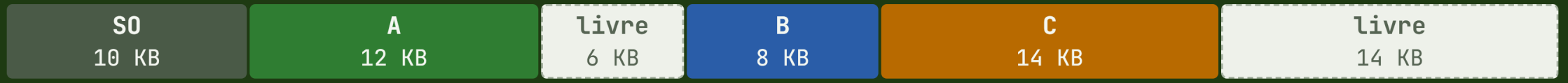
SISTEMAS OPERACIONAIS · CAPÍTULO 9

# Gerência de Memória

Como o sistema operacional divide a memória principal entre vários programas: alocação contígua, overlay, partições, estratégias de alocação e swapping.




Prof. Christiano Rossini · IFAL Campus Maceió · Bacharelado em Sistemas de Informação
Base: Machado & Maia, Arquitetura de Sistemas Operacionais, cap. 9

# Do programa único às partições e ao swapping

## Encontro 1 · Do programa único às partições

Por que gerenciar a memória

Funções básicas

Alocação contígua simples e proteção

Overlay

Partições estáticas e fragmentação interna

Exercício

## Encontro 2 · Partições dinâmicas e swapping

Partições dinâmicas e fragmentação externa

Junção de áreas livres e compactação

First-fit, best-fit e worst-fit

Swapping e relocação dinâmica

Laboratório: Windows e simulador

**Ponte com o capítulo 8:** o escalonador escolhe quem usa a UCP, mas só entre processos que estão na memória principal. Quantos cabem lá, e onde ficam, é decisão da gerência de memória.

# Para executar, o programa precisa estar na memória




## Memória principal

Onde ficam os programas em execução e seus dados. A UCP busca instruções e dados nela.

## Recurso disputado

Com multiprogramação, vários processos dividem a mesma memória. Quanto mais processos prontos na memória, menos a UCP fica ociosa.

## Por isso o SO gerencia a memória

Decide quem fica na memória, em que lugar, e o que vai para o disco quando falta espaço.

# O que a gerência de memória precisa garantir

### Manter o máximo de processos na memória

Mais processos prontos significa UCP menos ociosa e melhor uso do sistema.

### Executar programas maiores que a memória

Com técnicas como o overlay e, no próximo capítulo, a memória virtual.

### Proteger as áreas de memória

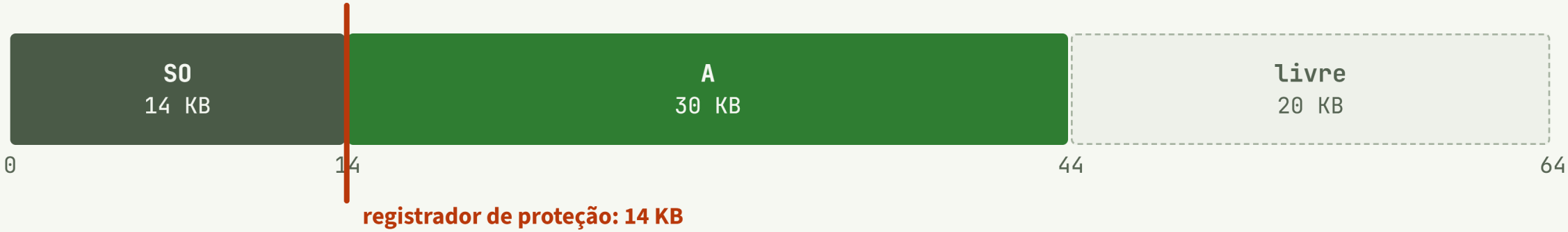
Um programa não pode acessar a área do SO nem a de outro processo.

### Usar o disco quando falta memória

Swapping: processos vão para o disco e voltam depois.

**Evolução das técnicas:** alocação contígua simples → overlay → particionada estática → particionada dinâmica → swapping.

# Um programa por vez: o SO e um usuário




### Simples

O SO fica no início da memória e o programa usa o restante. Fácil de implementar.

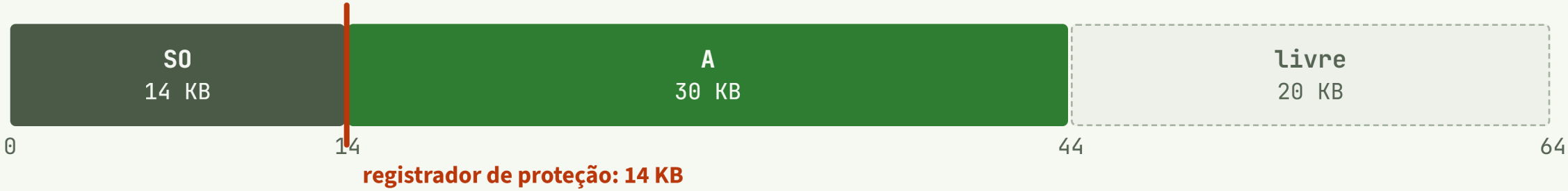
### Proteção

Cada endereço usado pelo programa é comparado ao registrador. Se invadir a área do SO, o hardware gera um erro e o SO encerra o programa.

### Subutilização

Um programa por vez: a área que ele não usa fica vazia, e a UCP fica parada enquanto ele espera E/S.

# Como o registrador de proteção funciona



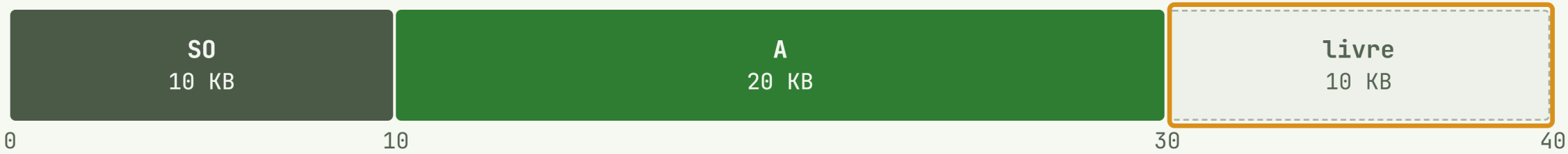

A acessa o endereço 20 KB → 20 ≥ 14 → **acesso permitido**

A acessa o endereço 10 KB → 10 < 14 → **interrupção: o SO encerra A**

## Quem pode mudar o registrador?

Só o sistema operacional: a instrução que altera o registrador é **privilegiada** e só executa em modo supervisor. Se o programa pudesse mudá-lo, a proteção não valeria nada. Para usar serviços do SO, o programa faz chamadas de sistema.

# Quanto da memória fica sem uso?

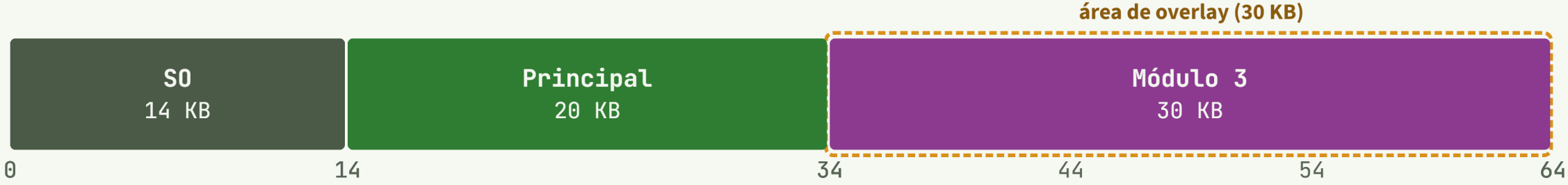



Memória de **40 KB**, SO de **10 KB** e um programa de **20 KB**. Qual a taxa de subutilização da memória principal?

Sobram 40 − 10 − 20 = **10 KB** sem uso: **25%** da memória principal (um terço da área do usuário). E, enquanto o programa espera uma E/S, a UCP também fica parada.

# Programa maior que a memória: overlay

Memória de 64 KB · SO de 14 KB · programa de 90 KB = módulo principal (20 KB) + módulos independentes de 10, 20 e 30 KB




Para caber, o programa é dividido em módulos e só o principal fica na memória; os demais são trazidos do disco para a área de overlay, que precisa ter o tamanho do maior módulo: 30 KB.

## Quem define os módulos

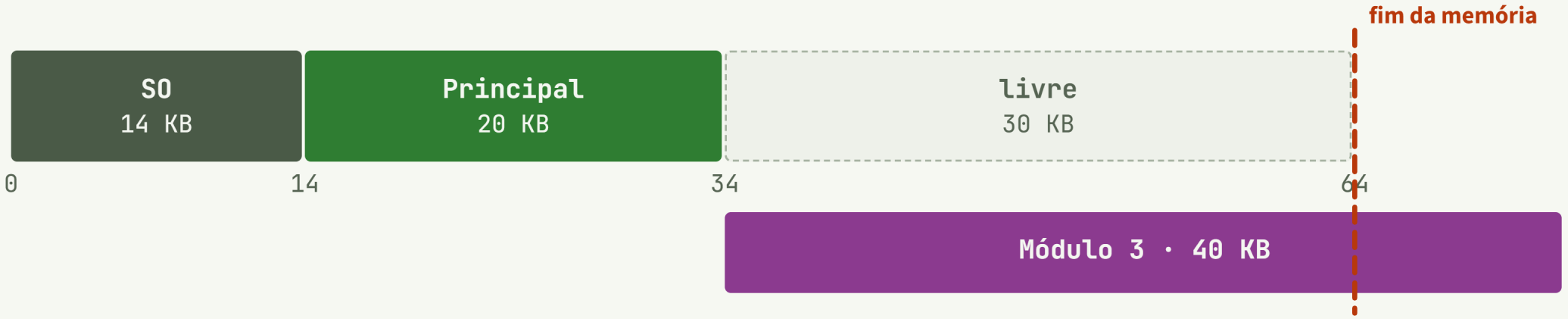
O programador divide o programa e decide quando cada módulo é carregado.

## Limitação

A área de overlay tem o tamanho do maior módulo; o programa fica mais complexo e cada troca exige carga do disco.

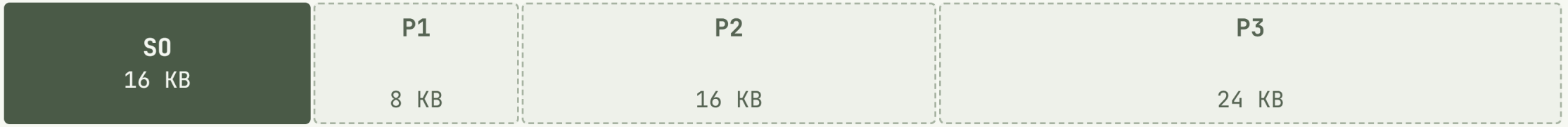
# E se o módulo 3 tivesse 40 KB?

Mesmo sistema (64 KB, SO de 14 KB, principal de 20 KB). O módulo 3 cresce de 30 KB para **40 KB**. O programa ainda roda só com overlay?




**Não:** 14 + 20 + 40 = 74 KB, mais que os 64 KB da memória. Saídas: dividir o módulo 3 em módulos menores (por exemplo, dois de 20 KB) ou reduzir o principal, para que tudo caiba na área disponível.

# Memória dividida em partições de tamanho fixo




### Definidas na inicialização

Número e tamanho das partições são fixos. Para mudar, é preciso reconfigurar e reiniciar o sistema.

### Tabela de partições

O SO guarda o início, o tamanho e a situação (livre ou ocupada) de cada partição.

### Multiprogramação

Vários programas na memória ao mesmo tempo, um em cada partição. A proteção usa registradores com os limites da partição.

# Absoluta ou relocável?

## ABSOLUTA: O PROGRAMA SÓ RODA NA PARTIÇÃO PARA A QUAL FOI COMPILADO




Se [illegible] para a partição livre

## RELOCÁVEL: O PROGRAMA PODE IR PARA QUALQUER PARTIÇÃO LIVRE EM QUE CAIBA




Se [illegible] da partição

**Relocável:** o código usa endereços relativos e é ajustado na carga para a partição escolhida. Mais flexível, mas o compilador e o carregador ficam mais complexos.

# Tabela de partições, filas e proteção

| Partição | Início | Tamanho | Situação |
|---|---|---|---|
| **P1** | 16 KB | 8 KB | ocupada (A, 6 KB) |
| **P2** | 24 KB | 16 KB | livre |
| **P3** | 40 KB | 24 KB | ocupada (C, 20 KB) |

## Filas de espera

**Absoluta:** uma fila por partição; cada programa espera a sua.

**Relocável:** uma fila única; o primeiro vai para qualquer partição livre em que caiba.

## Proteção

Dois registradores guardam o **limite inferior** e o **superior** da partição em execução.

C, na P3: limites 40 e 64 KB. Um acesso de C a 30 KB (P2) gera interrupção.

A cada troca de processo, o SO carrega nos registradores os limites da partição do processo que vai executar.

# Fragmentação interna: espaço perdido dentro da partição

Memória de 128 KB · SO de 64 KB · partições P1 = 8 KB, P2 = 24 KB e P3 = 32 KB




**Fragmentação interna total: 2 + 4 + 4 = 10 KB**

E se fossem **PA = 4 KB, PB = 16 KB e PC = 26 KB**? E **PA = 8 KB, PB = 24 KB e PC = 32 KB**?

b) 4 + 8 + 6 = **18 KB** perdidos. c) **0 KB**: cada programa ocupa a partição inteira. A perda depende de quanto o programa é menor que a partição em que foi colocado.

# Partições de 8, 24 e 32 KB, estática relocável

### a) Quatro programas ao mesmo tempo?

É possível executar quatro programas concorrentemente usando apenas a alocação particionada estática relocável?

### b) Um programa de 36 KB?

É possível executar um programa de 36 KB nesse sistema?

### Resposta a

Não. Há três partições, então no máximo três programas na memória. Um quarto só entraria trocando de lugar com outro (swapping), assunto do próximo encontro.

### Resposta b

Não. O maior programa cabe em 32 KB, o tamanho da maior partição. Seria preciso redefinir as partições (e reiniciar o sistema) ou usar overlay.

SÍNTESE DO ENCONTRO 1

# Cada técnica resolve um problema da anterior

| | |
|---|---|
| **Contígua simples** | um programa por vez, proteção por registrador, memória e UCP subutilizadas |
| **Overlay** | programa maior que a memória, com módulos definidos pelo programador |
| **Estática absoluta** | vários programas, mas cada um preso à sua partição |
| **Estática relocável** | qualquer partição livre em que o programa caiba; sobra a fragmentação interna |

Próximo encontro: partições dinâmicas, estratégias de alocação e swapping.

ENCONTRO 2

# Partições dinâmicas e swapping

Partições fixas desperdiçam memória dentro delas. E se cada programa recebesse exatamente o tamanho de que precisa?

# Cada programa recebe exatamente o que precisa




**E (20 KB) não cabe em nenhuma área livre, embora haja 28 KB livres no total: fragmentação externa**

## Sem partições fixas

Cada programa ocupa exatamente o seu tamanho: acaba a fragmentação interna.

## Fragmentação externa

Há 28 KB livres no total, mas divididos em áreas de 10 e 18 KB: nenhuma comporta E. São espaços livres pequenos e espalhados entre os programas.

# A lista de áreas livres




| Área livre | Início | Tamanho |
|---|---|---|
| **1** | 28 KB | 10 KB |
| **2** | 46 KB | 18 KB |

Total livre: **28 KB**
Maior área: **18 KB**

## Ao alocar

O SO escolhe uma área da lista, coloca o programa no início dela e deixa a sobra como uma área menor.

## Ao liberar

A área volta para a lista e é unida às áreas livres vizinhas, se houver.

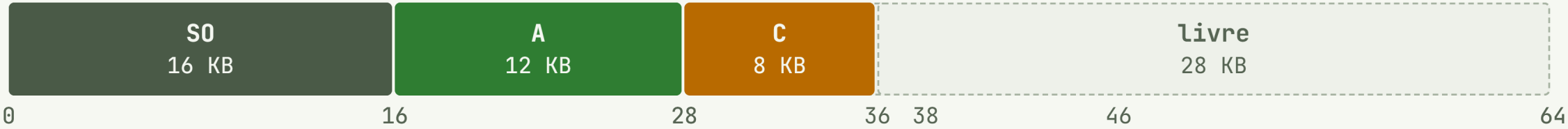
# Duas soluções para a fragmentação externa

## 1 · REUNIR ÁREAS LIVRES ADJACENTES




Áreas livres adjacentes são juntadas para formar uma única área livre maior.

## 2 · RELOCAÇÃO DOS PROGRAMAS (COMPACTAÇÃO)




Mesma ideia: juntar os 28 KB livres em uma única área.

**Custo da compactação:** os programas param enquanto são movidos, e o hardware precisa permitir relocação dinâmica. Por isso ela não é feita a todo momento.

# Onde colocar o programa? Três estratégias

ÁREAS LIVRES, EM ORDEM DE ENDEREÇO




**First-fit** — Escolhe a **primeira** área livre em que o programa cabe. A busca para assim que encontra: é a mais rápida. → 20 KB

**Best-fit** — Escolhe a **menor** área em que o programa cabe. Deixa a menor sobra possível. → 12 KB

**Worst-fit** — Escolhe a **maior** área livre. A sobra fica grande e ainda utilizável. → 20 KB

O SO mantém uma **lista de áreas livres**: ordenada por endereço para o first-fit, ou por tamanho para o best-fit e o worst-fit.

# Mesmas áreas livres, outros pedidos

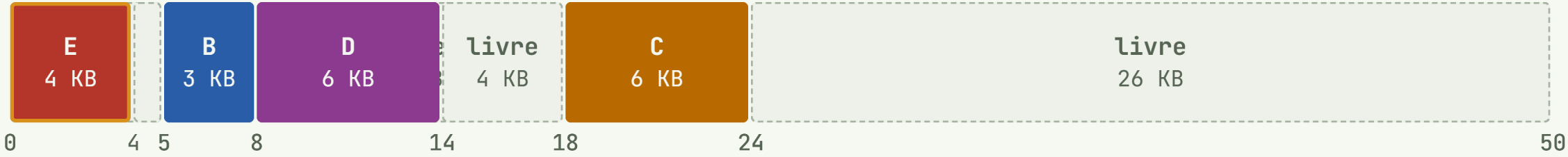
Áreas livres, na ordem: 10, 4, 20, 18, 7, 9, 12 e 15 KB. Qual área cada estratégia escolhe?

| Programa | First-fit | Best-fit | Worst-fit |
|---|---|---|---|
| **12 KB** | 20 KB | 12 KB | 20 KB |
| **10 KB** | 10 KB | 10 KB | 20 KB |
| **9 KB** | 10 KB | 9 KB | 20 KB |

O **best-fit** procura a área que deixa a menor sobra (9 KB cabe exato na área de 9). O **worst-fit** sempre vai para a maior área (20 KB). O **first-fit** fica com a primeira que serve, olhando da esquerda para a direita.

# First-fit: alocar D, liberar A, alocar E

Memória de 50 KB: A (5), B (3), livre (10), C (6), livre (26). Em ordem: alocar D (6 KB), liberar A, alocar E (4 KB).



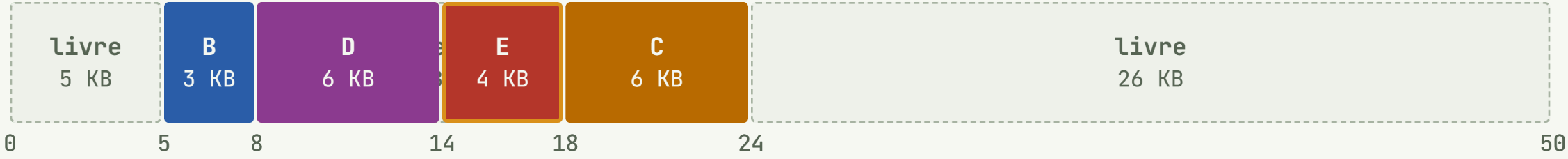

E (4 KB) alocado no endereço 0 KB, na área livre de 5 KB, sobram 1 KB livres.
Áreas livres: 1, 4, 26 KB.

## Resultado

E ficou na primeira área que serviu, a de 5 KB, deixando uma sobra de **1 KB**, pequena demais para quase qualquer programa.

# Best-fit: alocar D, liberar A, alocar E

Memória de 50 KB: A (5), B (3), livre (10), C (6), livre (26). Em ordem: alocar D (6 KB), liberar A, alocar E (4 KB).



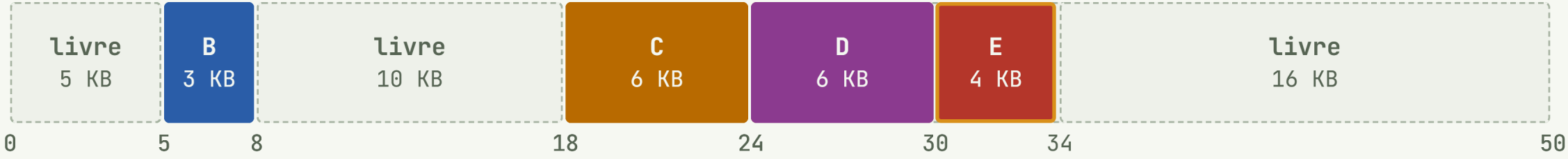

D (6 KB) alocado no endereço 8 KB, na área livre de 10 KB; sobram 4 KB livres.
Áreas livres: 4 e 26 KB.

## Resultado

E ficou na área de **4 KB**, sem nenhuma sobra. Mas, em geral, o best-fit tende a produzir sobras minúsculas.

# Worst-fit: alocar D, liberar A, alocar E

Memória de 50 KB: A (5), B (3), livre (10), C (6), livre (26). Em ordem: alocar D (6 KB), liberar A, alocar E (4 KB).




**E (4 KB) alocado no endereço 24 KB, na área livre de 26 KB; sobram 22 KB livres.**

Áreas livres: 5, 10 e 16 KB.

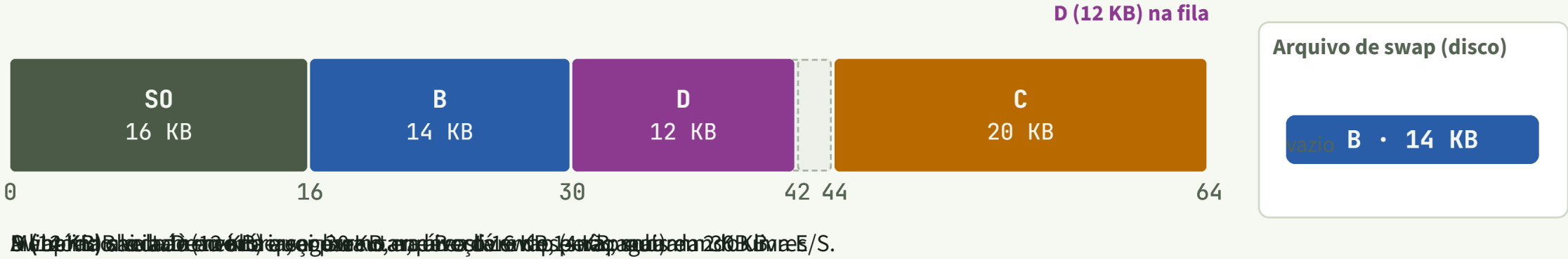
## Resultado

D e E foram para a maior área. As sobras continuam grandes (**16 KB**), mas a maior área livre vai sendo consumida.

# Comparando as estratégias

| Estratégia | Escolhe | Vantagem | Desvantagem |
|---|---|---|---|
| **First-fit** | a primeira área em que o programa cabe | busca rápida: para na primeira que serve | acumula sobras pequenas no início da memória |
| **Best-fit** | a menor área em que o programa cabe | deixa a menor sobra possível | cria áreas livres minúsculas e quase inúteis; percorre a lista toda |
| **Worst-fit** | a maior área livre | as sobras continuam grandes e utilizáveis | consome as áreas grandes: um programa grande pode não encontrar espaço |

# Swapping: quando a memória não basta




## Quem sai da memória

Em geral, um processo em espera ou de baixa prioridade. Quem decide é o SO, não o programador, como era no overlay.

## Custo

Cada swap é uma operação de E/S em disco, lenta. Se o sistema passa mais tempo trocando processos do que executando, o desempenho despenca.

# Quem sai, quando volta e o que fica

## Swap out: quem sai

Processos em **espera** (aguardando E/S ou um evento).

Processos de **baixa prioridade**.

Tirar um processo pronto de alta prioridade só atrasaria quem mais precisa executar.

## Swap in: quando volta

Quando já pode executar (a E/S que esperava terminou).

E quando há espaço livre suficiente.

Fora da memória, o processo não entra na fila de prontos.

## O que vai e o que fica

Vai para o arquivo de swap a **imagem inteira**: código, dados e pilha.

Fica na memória o **PCB**, na área do sistema: o SO sabe que o processo existe.

**Ponte com o capítulo 8:** a gerência de memória decide quantos e quais processos o escalonador terá na fila de prontos.

# Relocação dinâmica: o registrador base

**ANTES DO SWAP OUT: B EM 30 KB**




**DEPOIS DO SWAP IN: B EM 16 KB**




### O programa não muda

Os endereços do programa são relativos. Ao carregá-lo em outro lugar, o SO só troca o valor do registrador base.

### Por que o swapping precisa disso

Sem relocação dinâmica, B só poderia voltar exatamente para 30 KB e teria de esperar essa área ficar livre.

# Relocação a cada acesso, proteção e custos

| Relocação | Quando os endereços são ajustados | O processo pode mudar de lugar? |
|---|---|---|
| **Na carga (estática relocável)** | uma vez, quando o programa é carregado | não |
| **Dinâmica (registrador base)** | a cada acesso, pelo hardware | sim: basta trocar o registrador base |

### Registrador limite

Guarda o tamanho do processo. B tem 14 KB: o endereço lógico 15 KB é barrado, esteja B em 30 KB ou em 16 KB.

### Custo de cada troca

Cada swap lê ou grava o processo inteiro no disco, mesmo que ele só fosse usar uma parte do seu código.

**Thrashing:** com memória pequena demais, o sistema passa mais tempo trocando processos do que executando. Voltaremos a ele na memória virtual (cap. 10).

# Memória em uso, em cache e arquivo de paginação

EXEMPLO: COMPUTADOR COM 16 GB · MONITOR DE RECURSOS › MEMÓRIA




■ Em uso ■ Modificada ■ Em espera (cache) ■ Livre

## Gerenciador de Tarefas

Desempenho › Memória: em uso, disponível, confirmada e em cache.

Detalhes: memória de cada processo.

## Monitor de Recursos

Aba Memória: a barra acima. “Em espera” é cache de disco: o Windows a libera na hora se um programa pedir memória.

## Arquivo de paginação

pagefile.sys: área do disco usada quando a RAM não basta.

É o sucessor do swapping, que veremos no cap. 10 como paginação.

# As técnicas lado a lado

| Técnica | Programas na memória | Problema | Característica |
|---|---|---|---|
| **Contígua simples** | um | subutilização | proteção por registrador |
| **Overlay** | um | complexidade para o programador | programa maior que a memória |
| **Estática absoluta** | vários, um por partição | fragmentação interna | programa preso à sua partição |
| **Estática relocável** | vários, um por partição | fragmentação interna | qualquer partição livre que sirva |
| **Dinâmica** | vários | fragmentação externa | junção, compactação e estratégias |
| **Swapping** | mais do que cabem | custo de E/S | exige relocação dinâmica |

LABORATÓRIO

# Memória na prática

## No Windows

Gerenciador de Tarefas e Monitor de Recursos

Memória de cada processo com o PowerShell

Arquivo de paginação

## No simulador

Partições estáticas e fragmentação interna

First-fit, best-fit e worst-fit

Swapping e compactação

PARA LEVAR

# O que fica do capítulo 9

› A UCP só executa o que está na memória principal; o SO decide quem fica lá e onde.

› Partições fixas perdem memória dentro delas (fragmentação interna); partições dinâmicas, entre elas (fragmentação externa).

› First-fit é rápida, best-fit deixa sobras mínimas e worst-fit deixa sobras grandes.

› O swapping permite mais processos do que cabem, ao custo de E/S, e exige relocação dinâmica.

**Próximo capítulo: memória virtual.**